BTS SIO SLAM - PHP

Traitement des formulaires en PHP

Cours complet et détaillé - Fil rouge SupportDesk

Support de cours - à conserver avec le projet

Ce cours accompagne le diaporama « Traitement des formulaires en PHP ». Il reprend toutes les étapes du fil rouge SupportDesk et explique le rôle de chaque instruction importante. La recherche et le champ « bloquant » sont traités dans le document de suite et fin.

1. Objectifs et prérequis

- Envoyer un formulaire HTML vers PHP avec la méthode POST.
- Récupérer et valider les données avant tout enregistrement.
- Organiser les fonctions communes dans fonctions/fonctions.php.
- Lire et écrire des données MySQL avec mysqli.
- Utiliser une requête préparée pour séparer le SQL des valeurs.
- Afficher des données issues de la base sans exécuter du HTML ou du JavaScript injecté.
- Comprendre le cycle connexion -> requêtes -> récupération -> déconnexion -> HTML.

Prérequis - fonctions PHP, tableaux, conditions, foreach, bases SQL simples, notions de Git vues précédemment.

2. Organisation du projet

Fichier	Rôle
index.php	Afficher le formulaire de création d'un incident.
traitement.php	Recevoir le POST, valider, enregistrer puis rediriger.
liste.php	Récupérer et afficher tous les incidents.
fonctions/fonctions.php	Centraliser connexion, SELECT, INSERT et autres fonctions SQL.
css/style.css	Définir la présentation commune aux pages.
sql/base.sql	Créer la base SupportDesk et ses données de départ.

L'idée est de ne pas mélanger toutes les responsabilités dans un seul fichier. On reste encore en PHP procédural, mais on commence déjà à séparer le code réutilisable du HTML des pages.

3. Relier le HTML au CSS

Le fichier CSS n'est pas chargé automatiquement. Chaque page HTML qui doit utiliser les styles doit contenir un élément <link> dans son <head>. Le chemin est relatif au fichier PHP courant.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SupportDesk - Nouvel incident</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

Les classes du HTML doivent correspondre aux sélecteurs du CSS. Par exemple, class="conteneur" est relié à .conteneur dans style.css. Sans cette correspondance, la règle CSS ne s'applique pas.

```
<main class="conteneur">
  <nav class="navigation">...</nav>
  <form class="formulaire">...</form>
</main>
```

4. Construire le premier formulaire

Un formulaire HTML n'enregistre rien tout seul. Il prépare une requête HTTP qui sera envoyée à une autre page.

```
<form class="formulaire" action="traitement.php" method="post">
  <div class="champ">
    <label for="titre">Titre</label>
    <input type="text" id="titre" name="titre" required>
  </div>

  <div class="champ">
    <label for="description">Description</label>
    <textarea id="description" name="description" required></textarea>
  </div>

  <button class="bouton" type="submit">Créer</button>
</form>
```

Attribut	Rôle
action	Page qui recevra la requête lorsque l'utilisateur valide le formulaire.
method="post"	Envoie les valeurs dans le corps de la requête HTTP.
name	Nom de la donnée côté PHP : il devient une clé de $_POST.
required	Contrôle côté navigateur. Utile, mais insuffisant : PHP doit valider à nouveau.

Point important - un champ sans attribut name peut être visible à l'écran mais sa valeur n'est pas envoyée à PHP.

5. Ajouter la priorité

Le <select> envoie la valeur de l'option choisie. La valeur vide de la première option permet de détecter qu'aucun choix réel n'a été fait.

```
<div class="champ">
    <label for="priorite">Priorité</label>
    <select id="priorite" name="priorite" required>
        <option value="">-- Choisir --</option>
        <option value="basse">Basse</option>
        <option value="normale">Normale</option>
        <option value="haute">Haute</option>
    </select>
</div>
```

6. Refuser les mauvaises méthodes HTTP

traitement.php est une page de traitement : elle doit être appelée après l'envoi du formulaire en POST. Si quelqu'un saisit directement son URL, la requête est généralement un GET. On refuse alors le traitement.

```
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    // 405 = Method Not Allowed : la ressource existe,
    // mais la méthode HTTP utilisée n'est pas autorisée ici.
    http_response_code(405);
    exit("Méthode non autorisée.");
}
```

http_response_code(405) ne remplace pas exit(). Le premier fixe le statut HTTP envoyé au navigateur ; le second arrête réellement le script PHP.

7. Récupérer les valeurs reçues

```
$titre = trim($_POST["titre"] ?? "");
$description = trim($_POST["description"] ?? "");
$priorite = $_POST["priorite"] ?? "";

$utilisateurId = (int) ($_POST["utilisateur_id"] ?? 0);
$categorieId = (int) ($_POST["categorie_id"] ?? 0);
```

Écriture	Explication
$_POST["titre"]	Valeur envoyée par le champ name="titre".

Écriture	Explication
?? ""	Valeur par défaut si la clé n'existe pas.
trim(...)	Retire les espaces en début et fin de chaîne.
(int)	Convertit explicitement une valeur vers un entier.

Attention - toutes les données venant du navigateur doivent être considérées comme non fiables, même si le formulaire HTML contient required, select ou des types spécifiques.

8. Valider côté serveur

On accumule les erreurs dans un tableau au lieu d'arrêter au premier problème. L'utilisateur peut ainsi corriger plusieurs champs en une seule fois.

```
$erreurs = [];

if ($titre === "") {
    $erreurs[] = "Le titre est obligatoire.";
}

if ($utilisateurId <= 0) {
    $erreurs[] = "Utilisateur invalide.";
}

if ($categorieId <= 0) {
    $erreurs[] = "Catégorie invalide.";
}
```

Pour la priorité, on contrôle une liste blanche : seules les valeurs prévues sont autorisées.

```
$prioritesAutorisees = ["basse", "normale", "haute"];

if (!in_array($priorite, $prioritesAutorisees, true)) {
    $erreurs[] = "Priorité invalide.";
}
```

Le troisième paramètre true demande une comparaison stricte : PHP compare la valeur et le type.

```
if (count($erreurs) > 0) {
    foreach ($erreurs as $erreur) {
        echo "<p>" . htmlspecialchars($erreur, ENT_QUOTES, "UTF-8") . "</p>";
    }
    exit;
}
```

9. Comprendre la base relationnelle

SupportDesk utilise trois tables. L'incident ne recopie pas le nom de l'utilisateur ni le libellé de la catégorie : il stocke leurs identifiants.

Table	Colonnes principales	Rôle
utilisateur	id, nom, email	Personne qui déclare l'incident.
categorie	id, libelle	Famille de l'incident.
incident	id, titre, description, priorite, utilisateur_id, categorie_id, date_creation	Incident enregistré et liens vers les deux autres tables.

Les clés étrangères empêchent normalement de rattacher un incident à un utilisateur ou une catégorie qui n'existe pas.

10. Centraliser la connexion MySQL

Le fichier fonctions/fonctions.php contient les fonctions réutilisables. Les pages l'incluent avec require_once. __DIR__ représente le dossier du fichier courant ; cela rend le chemin plus fiable.

```
require_once __DIR__ . "/fonctions/fonctions.php";
```

fonction connexion()

```
function connexion(): ?mysqli
{
    $con = mysqli_connect(
        "localhost",
        "root",
        "",
        "supportdesk"
    );

    if ($con === false) {
        return null;
    }

    mysqli_set_charset($con, "utf8mb4");
    return $con;
}
```

Le type ?mysqli signifie que la fonction peut renvoyer soit un objet/handle mysqli valide, soit null. utf8mb4 permet les caractères UTF-8 complets.

fonction deconnexion()

```
function deconnexion(?mysqli $con): void
{
```

```
    if ($con !== null) {
        mysqli_close($con);
    }
}
```

On ferme la connexion après la dernière requête utile de la page. PHP la fermerait automatiquement en fin de script, mais la fermeture explicite libère la ressource plus tôt et rend le cycle plus compréhensible.

11. Lire les utilisateurs et catégories

```
function recupererCategories(mysqli $con): array
{
    $sql = "SELECT id, libelle FROM categorie ORDER BY libelle";
    $resultat = mysqli_query($con, $sql);

    if ($resultat === false) {
        return [];
    }

    return mysqli_fetch_all($resultat, MYSQLI_ASSOC);
}
```

mysqli_fetch_all(..., MYSQLI_ASSOC) transforme toutes les lignes SQL en tableau PHP associatif. Après cette étape, les données vivent dans une variable PHP : la connexion MySQL n'est plus nécessaire pour les parcourir.

12. Préparer les listes avant le HTML

```
$con = connexion();

if ($con === null) {
    exit("Connexion impossible à la base de données.");
}

$utilisateurs = recupererUtilisateurs($con);
$categories = recupererCategories($con);

deconnexion($con);
```

La déconnexion ne supprime pas $utilisateurs ni $categories. Ces variables contiennent déjà des tableaux PHP. On peut donc fermer MySQL puis générer le formulaire.

13. Générer un <select> dynamique

```
<select id="utilisateur_id" name="utilisateur_id" required>
    <option value="">-- Choisir un utilisateur --</option>

    <?php foreach ($utilisateurs as $utilisateur): ?>
        <option value="<?= (int) $utilisateur["id"] ?>">
            <?= htmlspecialchars($utilisateur["nom"], ENT_QUOTES, "UTF-8") ?>
        </option>
```

```
    <?php endforeach; ?>
</select>
```

La syntaxe foreach (...): ... endforeach; est équivalente à foreach (...) { ... }. Elle est souvent plus lisible dans une page où PHP est interrompu par beaucoup de HTML. Dans un bloc presque entièrement PHP, les accolades restent très naturelles.

14. Insérer avec une requête préparée

Une requête préparée évite de fabriquer une commande SQL en concaténant directement des valeurs utilisateur. Elle sépare la structure SQL des données.

```
$sql = "INSERT INTO incident
        (titre, description, priorite, utilisateur_id, categorie_id)
        VALUES (?, ?, ?, ?, ?)";

$stmt = mysqli_prepare($con, $sql);

mysqli_stmt_bind_param(
    $stmt,
    "sssii",
    $titre,
    $description,
    $priorite,
    $utilisateurId,
    $categorieId
);

return mysqli_stmt_execute($stmt);
```

Étape	Ce qui se passe
PREPARE	MySQL prépare la structure de la commande avec des ? à la place des valeurs.
BIND	PHP associe les variables aux ? et indique leur type : s pour string, i pour integer.
EXECUTE	MySQL exécute la requête avec les valeurs liées comme données.

Pourquoi ? - la saisie utilisateur reste une donnée et ne devient pas un morceau de la commande SQL. C'est la base de la prévention des injections SQL.

15. Enregistrer puis rediriger

```
$con = connexion();
```

```
$ok = ajouterIncident(
    $con,
    $titre,
    $description,
    $priorite,
    $utilisateurId,
    $categorieId
);

// L'INSERT est terminé : aucune autre requête n'est nécessaire.
deconnexion($con);

if ($ok) {
    header("Location: liste.php?creation=ok");
    exit;
}
```

La redirection évite de rester sur traitement.php et permet d'afficher immédiatement la liste actualisée. Le paramètre creation=ok sert uniquement à afficher un message de confirmation.

16. Récupérer les incidents avec des jointures

La table incident contient des identifiants. Pour afficher des informations lisibles, on rejoint utilisateur et categorie.

```
SELECT
    i.id,
    i.titre,
    i.description,
    i.priorite,
    i.date_creation,
    u.nom AS utilisateur,
    c.libelle AS categorie
FROM incident i
INNER JOIN utilisateur u ON u.id = i.utilisateur_id
INNER JOIN categorie c ON c.id = i.categorie_id
ORDER BY i.date_creation DESC, i.id DESC
```

Les alias i, u et c raccourcissent l'écriture. AS utilisateur et AS categorie donnent des noms de clés pratiques dans le tableau PHP retourné.

17. Récupérer avant de générer le HTML

```
$con = connexion();
$incidents = recupererIncidents($con);

// Toutes les lignes sont maintenant dans un tableau PHP.
deconnexion($con);

?>
<!DOCTYPE html>
```

Fermer avant le HTML n'est pas une obligation technique. C'est un choix propre ici parce que toutes les requêtes sont terminées. Le tableau $incidents reste disponible pour le reste du script.

18. Afficher et échapper les données

```
<?php foreach ($incidents as $incident): ?>
  <article class="incident">
    <h2>
      <?= htmlspecialchars($incident["titre"], ENT_QUOTES, "UTF-8") ?>
    </h2>

    <p class="incident-description">
      <?= nl2br(htmlspecialchars(
        $incident["description"],
        ENT_QUOTES,
        "UTF-8"
      )) ?>
    </p>
  </article>
<?php endforeach; ?>
```

Échapper au moment de l'affichage signifie convertir les caractères qui ont un sens en HTML (<, >, &, guillemets...) en représentation textuelle. Le navigateur les affiche au lieu de les interpréter comme des balises.

On ne modifie pas systématiquement la donnée avant de l'enregistrer : la même donnée pourrait plus tard être affichée dans un autre contexte (JSON, CSV, texte, API). On protège la sortie selon son contexte. Ici, le contexte est HTML.

nl2br() transforme les retours à la ligne en
. On applique d'abord htmlspecialchars(), puis nl2br(), afin que le contenu utilisateur soit neutralisé avant l'ajout des balises
 générées par PHP.

19. Test XSS local

Une XSS (Cross-Site Scripting) se produit lorsqu'une donnée contrôlée par un utilisateur est réaffichée comme du HTML ou du JavaScript exécutable. Dans l'application locale de cours, saisir dans la description :

```
<script>alert("XSS")</script>
```

Avec un affichage vulnérable tel que echo $incident["description"]; le navigateur peut interpréter la balise. Avec htmlspecialchars(), la chaîne apparaît comme du texte et le script ne s'exécute pas.

Règle - valider les données à l'entrée ; utiliser des requêtes préparées pour le SQL ; encoder/échapper les données au moment de la sortie HTML.

20. Enregistrer une version fonctionnelle avec Git

```
git status
git add .
git commit -m "Formulaires PHP - validation et BDD"
git push
```

Avant le commit, vérifier que le formulaire crée réellement un incident, que la liste s'actualise et que le test XSS s'affiche comme du texte.

21. Synthèse

Étape	Question à se poser
Formulaire	Quels noms de champs seront envoyés ?
Réception	La page accepte-t-elle la bonne méthode HTTP ?
Validation	Les valeurs sont-elles présentes, autorisées et cohérentes ?
BDD	La requête sépare-t-elle bien SQL et données ?
Lecture	Les jointures ramènent-elles des informations lisibles ?
Affichage	Les données utilisateur sont-elles échappées pour le HTML ?
Git	La version poussée fonctionne-t-elle réellement ?

La prochaine étape terminera le fil rouge procédural avec la recherche préparée et l'évolution « incident bloquant ». Ensuite seulement, l'application servira de point de comparaison pour introduire la POO et PDO.